Polytechnic
University
of Coimbra

Python Language Programming

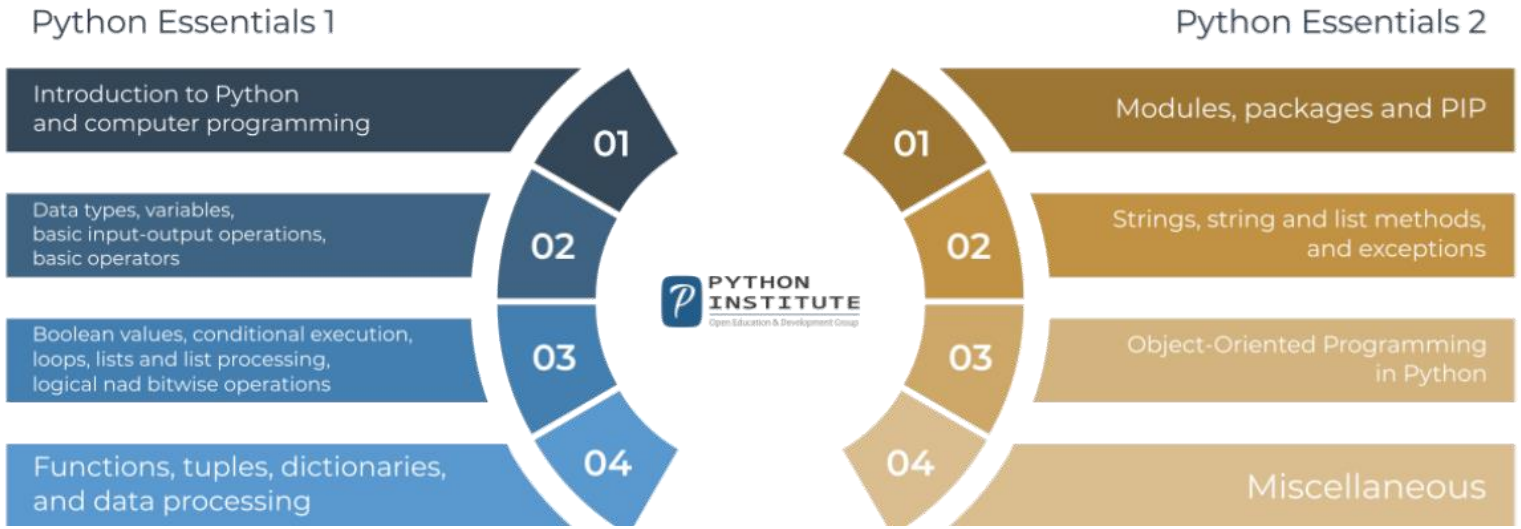
Lists, Tuples, Dictionaries

Polytechnic University of Coimbra

Python Language Programming

Python INSTITUTE
Open Education & Development Group

- **Disclaimer**
  - These slides and notes are based on the contents of the courses:
    - **Python Essentials 1 and 2**
      - aligned with the PCEP and PCAP certifications, respectively;
      - developed by the OpenEDG Python Institute; (www.pythoninstitute.org)
      - offered in the Cisco Networking Academy program.

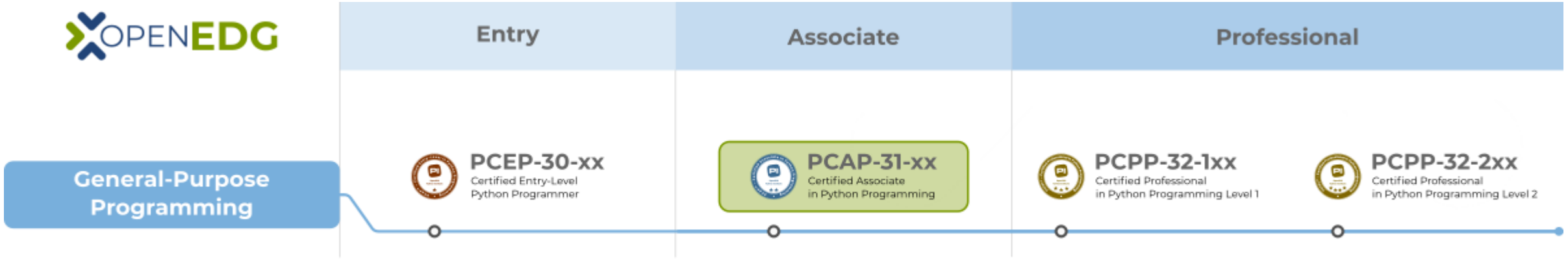


Polytechnic University of Coimbra

Python Language Programming

Python INSTITUTE
Open Education & Development Group

- **Disclaimer**
  - These slides and notes are based on the contents of the courses:
    - **Python Essentials 1 and 2**
      - aligned with the PCEP and PCAP certifications, respectively;
      - developed by the OpenEDG Python Institute; (www.pythoninstitute.org)
      - offered in the Cisco Networking Academy program.



OpenEDG Python Institute Certification Roadmap for the General-Purpose Programming track

Python INSTITUTE

Polytechnic University of Coimbra

Table of Contents

Polytechnic University of Coimbra

Python Language Programming

- **Lists**

```
list_example=[5.6, 7, [4, 7], False, "Hello"]
```

- a type of variable used to store multiple objects
- ordered and mutable collection of comma-separated items between square brackets
- can contain objects of different types
  - python-like lists do not exist in C

C	Python
- Arrays CANNOT contain different types - Arrays CANNOT change their size	- Lists can contain different objects - Lists can change their dimension

```
array_example=[67, 7, 9]
```

```
list_example=[5.6, 7, [4, 7], False]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Accessing list content**
    - Each of the list's elements may be accessed separately

```
numbers = [10, 5, 7, 2, 1]
print(numbers[1]) # Accessing the list’s second element.
```

5

    - The list may also be printed as a whole

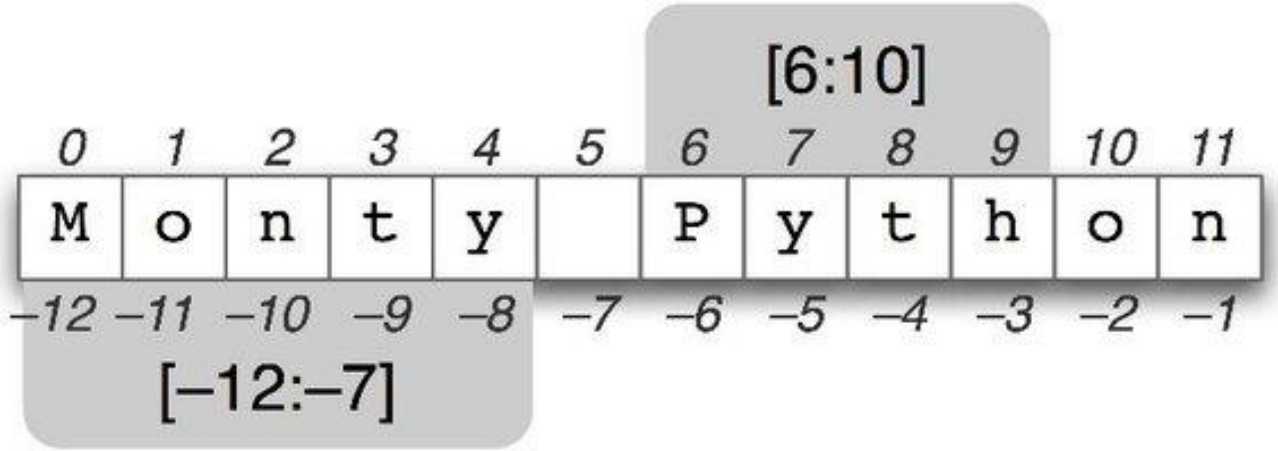
```
numbers = [10, 5, 7, 2, 1]
print("Original list content:", numbers)
```

Original list content: [10, 5, 7, 2, 1]

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Accessing list content**
    - Negative indices are legal



https://www.quora.com/What-are-negative-indexes-and-why-are-they-used-in-Python

```
numbers = [111, 7, 2, 1]
print(numbers[-1])
print(numbers[-2])
```

```
1
2
```

Python Language Programming

- **Lists**
  - **Accessing list content**
    - To traverse a list we can:
      - use a for-loop like in C
      - iterate directly on the list

```
my_list = [10, 1, 8, 3, 5]
total = 0

for i in range(len(my_list)):
    total += my_list[i]

print(total)
```

27

```
my_list = [10, 1, 8, 3, 5]
total = 0

for i in my_list:
    total += i

print(total)
```

27

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Update**

```
my_list = [1, None, True, 'I am a string', 256, 0]
print(my_list[3])
print(my_list[-1])

my_list[1] = '?'
print(my_list)
```

```
I am a string
0
[1, '?', True, 'I am a string', 256, 0]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Insertion**
    - **`append(val)`**
      - insert the element `val` at the end of the list

```
lista = ['qwe', 'asd', 'zxc']
lista.append(25)
print(lista)
```

```
['qwe', 'asd', 'zxc', 25]
```

    - **`insert(index, val)`**
      - insert an element in the position index

```
lista.insert(0, 14)
print(lista)
```

```
[14, 'qwe', 'asd', 'zxc', 25]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Insertion**

```
numbers = [111, 7, 2, 1]
print(len(numbers))
print(numbers)

numbers.append(4)

print(len(numbers))
print(numbers)

numbers.insert(0, 222)
print(len(numbers))
print(numbers)
```

```
4
[111, 7, 2, 1]
5
[111, 7, 2, 1, 4]
6
[222, 111, 7, 2, 1, 4]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Insertion**

```
my_list = []

for i in range(5):
    my_list.append(i + 1)

print(my_list)
```

```
[1, 2, 3, 4, 5]
```

```
my_list = []

for i in range(5):
    my_list.insert(0, i + 1)

print(my_list)
```

```
[5, 4, 3, 2, 1]
```

Polytechnic University of Coimbra

Python Language Programming

- Lists
  - Insertion
    - swap the values of two variables

```
variable_1 = 1
variable_2 = 2

auxiliary = variable_1
variable_1 = variable_2
variable_2 = auxiliary
```

=

```
variable_1 = 1
variable_2 = 2

variable_1, variable_2 = variable_2, variable_1
```

Polytechnic University of Coimbra

Python Language Programming

- Lists
  - Insertion
    - swap the list's elements to reverse their order

```
my_list = [10, 1, 8, 3, 5]

my_list[0], my_list[4] = my_list[4], my_list[0]
my_list[1], my_list[3] = my_list[3], my_list[1]

print(my_list)
```

```
[5, 3, 8, 1, 10]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Insertion**
    - **swap the list's elements to reverse their order**

```
my_list = [10, 1, 8, 3, 5]
length = len(my_list)

for i in range(length // 2):
    my_list[i], my_list[length - i - 1] = my_list[length - i - 1], my_list[i]

print(my_list)
```

```
[5, 3, 8, 1, 10]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Deletion**
    - **`del lista[index]`**
      - delete the element in the position index of the list
      - it's an instruction, not a function

```
numbers = [10, 5, 7, 2, 1]
print(len(numbers))
print(numbers)

del numbers[1]
print(len(numbers))
print(numbers)
```

```
5
[10, 5, 7, 2, 1]
4
[10, 7, 2, 1]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Deletion**
    - **`pop(index)`** remove the element index

```
lista = [14, 'qwe', 'asd', 'zxc', 25]
lista.pop(1)
print(lista)
```

```
[14, 'asd', 'zxc', 25]
```

    - **`remove(val)`** remove the first occurrence of val

```
lista = [14, 'asd', 'zxc', 25]
lista.append("zxc")
print(lista)
lista.remove("zxc")
print(lista)
```

```
[14, 'asd', 'zxc', 25, 'zxc']
[14, 'asd', 25, 'zxc']
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **List Name Meaning**
    - Problem:
      - the <u>name of a list represents the memory location</u> where the list is stored

```
list_1 = [1]
list_2 = list_1
list_1[0] = 2
print(list_2)
```

```
2
```

l2 = l1 does not make a copy of the l1 list. Instead makes the variables l1 and l2 point to one and the same list in memory.
Modifying one of them affects the other, and vice versa.

    - Solution:

```
list_2 = list_1[:]
```

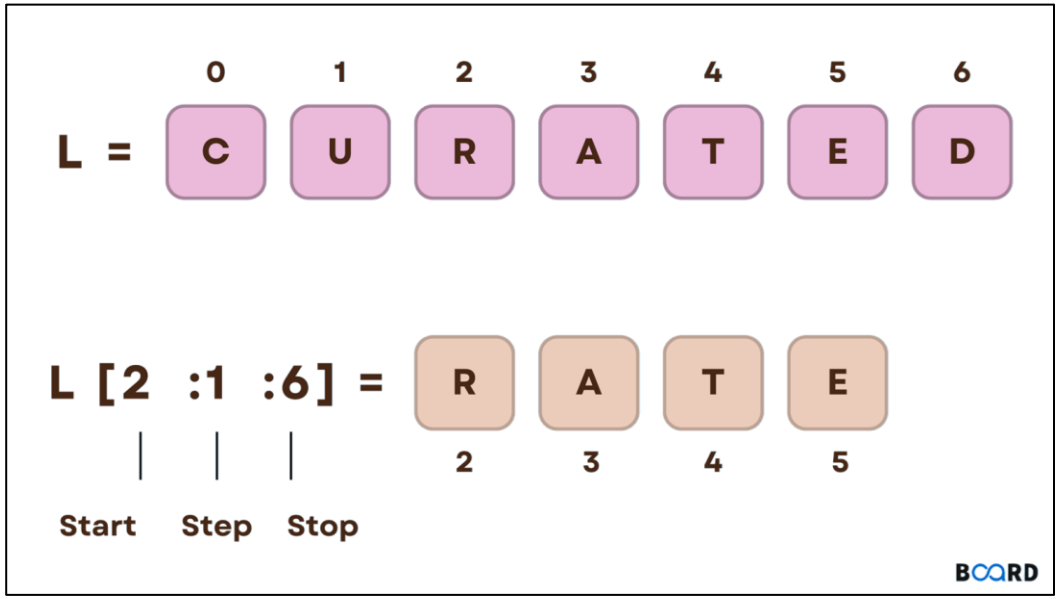
Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Slicing**
    - process of accessing a subset of a list for some action
    - the list slice is a list

```
list_name[start:stop:steps]
```

    - list slicing can be done by passing:
      - the start or stop parameters
      - the start and stop parameters
      - start, stop, and steps parameters



https://www.boardinfinity.com/blog/list-slicing-in-python/

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Slicing**

```
my_list = [1, 2, 3, 4, 5]
del my_list[0:2]
print(my_list)
del my_list[:] # deletes the list content
print(my_list)
```

[3, 4, 5]
[ ]

Polytechnic University of Coimbra

Python Language Programming

https://www.linkedin.com/pulse/cheat-sheet-python-list-comprehension-nicholas-key/

Index from right	-9	-8	-7	-6	-5	-4	-3	-2	-1
Index from left	0	1	2	3	4	5	6	7	8
	A	B	C	D	E	F	G	H	I

Frequency of usage: VERY REGULAR

```
list[-4]          = 'F'
list[4]           = 'E'
list[:]           = ['A', 'B', 'C', 'D', 'E', 'F',
                     'G', 'H', 'I']
list[-4:]         = ['F', 'G', 'H', 'I']
list[4:]          = ['E', 'F', 'G', 'H', 'I']
list[:4]          = ['A', 'B', 'C', 'D']
list[:-4]         = ['A', 'B', 'C', 'D', 'E']
list[-12:1]       = ['A']
list[-12:3]       = ['A', 'B', 'C']
list[-12:-3]      = ['A', 'B', 'C', 'D', 'E', 'F']
list[1:-8]        = []
list[1:-11]       = []
list[2:6]         = ['C', 'D', 'E', 'F']
list[-7:6:2]      = ['C', 'E']
list[2:7:4]       = ['C', 'G']
list[2:7:5]       = ['C']
list[3:8:2]       = ['D', 'F', 'H']
```

Frequency of usage: REGULAR

```
list[-1::]        = ['I']
list[-3::]        = ['G', 'H', 'I']
list[3::]         = ['D', 'E', 'F', 'G', 'H', 'I']
list[::-1]        = ['I', 'H', 'G', 'F', 'E', 'D',
                     'C', 'B', 'A']
list[::-4]        = ['I', 'E', 'A']
list[::2]         = ['A', 'C', 'E', 'G', 'I']
list[::4]         = ['A', 'E', 'I']
```

Frequency of usage: ONCE IN A WHILE

```
list[-7::2]       = ['C', 'E', 'G', 'I']
list[-7::3]       = ['C', 'F', 'I']
list[1::-1]       = ['B', 'A']
list[1::-2]       = ['B']
list[1::-7]       = ['B']
list[6::-3]       = ['G', 'D', 'A']
```

ESTGOH

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **`in` and `not` in operators**

```
elem in my_list
elem not in my_list
```

  - **`in`:** returns True if a given element is currently stored somewhere inside the list
  - **`not in`**: returns True if a given element is absent in a list

```
my_list = [0, 3, 12, 8, 2]

print(5 in my_list)
print(5 not in my_list)
print(12 in my_list)
```

False
True
True

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **List concatenation**
    - **operator +**
      - concatenate two lists

```
list_1 = [1, 2, 3]
list_2 = [4, 5, 6]
print(list_1  + list_2)
```

```
[1, 2, 3, 4, 5, 6]
```

    - **operator ***
      - replicate a list N times

```
list_1 = [0]*10
print(list_1)
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **List comprehension**
    - allows you to create new lists from existing ones in a concise and elegant way

```
list = [expression for element in list if conditional]
```

```
cubed = [num ** 3 for num in range(5)]
print(cubed)
```

```
[0, 1, 8, 27, 64]
```

```
cubed = [[i ** 3 for i in range(5)] for j in range(2)]
print(cubed)
```

```
[[0, 1, 8, 27, 64], [0, 1, 8, 27, 64]]
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Lists of Lists**
    - A list can be defined inside another list
    - To access an element inside we have to access the first level list, and then the element inside it
    - The **outer list** contains the rows of the matrix, the **inner ones** the elements of the columns related to that row

```
list_1 = [4.5, 6.3, 7.2]
list_2 = [[1, 2, 3], [“a”, “b”, “c”], [True, False], list_1]
print(list_2[1])
print(list_2[1][2])
```

```
[‘a’, ‘b’, ‘c’]
c
```

Polytechnic University of Coimbra

Python Language Programming

- **Lists**
  - **Lists of Lists**

```
temps = [[0.0 for h in range(24)] for d in range(31)]
#
# The matrix will be updated here.
#

total = 0.0

for day in temps:
    total += day[11]

average = total / 31

print("Average temperature at noon:", average)
```

Average temperature at noon: 0.0

Polytechnic University of Coimbra

Python Language Programming

- **Tuples**
  - an immutable sequence type - cannot change their elements (i.e. cannot append tuples, or modify, or remove tuple elements)
  - tuples use parenthesis, whereas list use brackets
    - possible to create a tuple from a set of values separated by commas

```
tuple_1 = (1, 2, 4, 8)
tuple_2 = 1., .5, .25, .125
print(tuple_1)
print(tuple_2)
```

```
(1, 2, 4, 8)
(1.0, 0.5, 0.25, 0.125)
```

```
one_element_tuple_1 = (1, )
one_element_tuple_2 = 1.,
```

Note: a tuple has to be distinguishable from an ordinary, single value. This way a one element tuple must end with a comma!

Polytechnic University of Coimbra

Python Language Programming

- **Tuples**
  - tuple's elements can be variables, not only literals

```
var = 123
t1 = (1, )
t2 = (2, )
t3 = (3, var)
t1, t2, t3 = t2, t3, t1
print(t1, t2, t3)
```

```
(2,) (3, 123) (1,)
```

Polytechnic University of Coimbra

Python Language Programming

- **Tuples**
  - **Example**

```
my_tuple = (1, 10, 100)

t1 = my_tuple + (1000, 10000)
t2 = my_tuple * 3

print(len(t2))
print(t1)
print(t2)
print(10 in my_tuple)
print(-10 not in my_tuple)
```

```
9
(1, 10, 100, 1000, 10000)
(1, 10, 100, 1, 10, 100, 1, 10, 100)
True
True
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - a mutable sequence type
  - a set of key-value pairs (list contains a set of numbered values).
    - key must be unique and immutable (integer, float, string, but not a list)

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}
phone_numbers = {'boss': 5551234567, 'Suzy': 22657854310}
empty_dictionary = {}

print(dictionary)
print(phone_numbers)
print(empty_dictionary)
```

```
{'cat': 'chat', 'dog': 'chien', 'horse': 'cheval'}
{'boss': 5551234567, 'Suzy': 22657854310}
{}
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - **Accessing Data**
    - values can be accessed by placing the key within square brackets next to the dictionary.

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}
phone_numbers = {'boss': 5551234567, 'Suzy': 22657854310}

print(dictionary["dog"])
print(phone_numbers["Suzy"])
```

```
chien
22657854310
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - **Accessing Data**
    - **`keys()`**
      - the method returns all the **keys** gathered within the dictionary

```
dictionary = {"dog": "chien", "cat": "chat", "horse": "cheval"}

for key in sorted(dictionary.keys()):
    print(key, "->", dictionary[key])
```

```
cat -> chat
dog -> chien
horse -> cheval
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - **Accessing Data**
    - **`items()`**
      - the method returns tuples where each tuple is a **key-value pair**

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}

for english, french in dictionary.items():
    print(english, "->", french)
```

```
cat -> chat
dog -> chien
horse -> cheval
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - **Accessing Data**
    - **`values()`**
      - the method returns all the **values** gathered within the dictionary

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}

for french in dictionary.values():
    print(french)
```

```
chat
chien
cheval
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - **Add, Modify and Remove Data**

```
dictionary = {"cat": "chat", "dog": "chien", "horse": "cheval"}

dictionary['cat'] = 'minou' # update value
print(dictionary).

dictionary['swan'] = 'cygne' # add value
print(dictionary)

# update() method merges two dictionaries
dictionary.update({"bird": "duck"}) # add value
print(dictionary)

del dictionary['dog'] # remove value
print(dictionary)

dictionary.pop('cat') # remove value
print(dictionary)
```

Polytechnic University of Coimbra

Python Language Programming

- **Dictionaries**
  - **Add, Modify and Remove Data**

...

```
{'cat': 'minou', 'dog': 'chien', 'horse': 'cheval'}
{'cat': 'minou', 'dog': 'chien', 'horse': 'cheval', 'swan': 'cygne'}
{'cat': 'minou', 'dog': 'chien', 'horse': 'cheval', 'swan': 'cygne', 'bird': 'duck'}
{'cat': 'minou', 'horse': 'cheval', 'swan': 'cygne', 'bird': 'duck'}
{'horse': 'cheval', 'swan': 'cygne', 'bird': 'duck'}
```

Polytechnic University of Coimbra

Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved September 4, 2024, from https://www.netacad.com/**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.